



## Week 05 Task-14

### Cross-Site Scripting (XSS) -DVWA

CandidateName: Asad Ali

Reference Number: TBS-INT-2026-445

Submission Date: 17/9/2026

Submitted To: TechBiz Security

## Table of Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1) Executive Summary .....</b>                              | <b>3</b>  |
| <b>2) Scope of Task .....</b>                                  | <b>3</b>  |
| <b>3) Tools Used .....</b>                                     | <b>3</b>  |
| <b>4) Explaining XSS .....</b>                                 | <b>3</b>  |
| <b>5) Types of XSS .....</b>                                   | <b>3</b>  |
| 5.1) Reflected XSS .....                                       | 3         |
| 5.2) Stored XSS .....                                          | 4         |
| <b>6) Reflected XSS Practical .....</b>                        | <b>5</b>  |
| <b>7) Stored XSS Practical .....</b>                           | <b>7</b>  |
| <b>8) Difference between Reflected XSS &amp; Stored XSS ..</b> | <b>9</b>  |
| <b>9) Impact of XSS .....</b>                                  | <b>10</b> |
| <b>10) Remediation of XSS .....</b>                            | <b>10</b> |
| <b>11) TryHackMe Room XSS Practical .....</b>                  | <b>11</b> |
| <b>12) Conclusion .....</b>                                    | <b>11</b> |

# 1) Executive Summary

This task focused on understanding and demonstrating Cross-Site Scripting (XSS), a common web application vulnerability that occurs when user input is not properly validated, sanitized, or encoded, allowing attackers to inject malicious scripts into web pages viewed by other users. The task covered the identification of both Reflected and Stored XSS, their potential impact on web applications and user browsers, and the security measures such as output encoding and input validation used to prevent such vulnerabilities.

## 2) Scope of Task

The scope of this task was limited to the identification and demonstration of Cross-Site Scripting (XSS) vulnerabilities in the DVWA application within an authorized home network environment. Testing was restricted on DVWA to Reflected XSS (non-persistent injection via input fields) and Stored XSS (persistent injection via database-backed guestbook features) to understand how unvalidated user inputs can be manipulated to execute malicious scripts in user browsers. The TryHackMe XSS room covered comprehensive practical exercises and theoretical concepts of XSS. This task also covered the potential impact of XSS such as session hijacking, credential theft, and account takeover and the remediation measures used to prevent such vulnerabilities. No real-world or unauthorized targets were tested. All testing is performed on an authorized network.

## 3) Tools Used

- Kali Linux
- Damn Vulnerable Web Application (DVWA)

## 4) Explaining XSS

**Cross-Site Scripting (XSS)** is a common web application vulnerability that allows attackers to inject malicious scripts into trusted websites. It occurs when an application fails to properly validate, sanitize, or encode user input, causing the script to execute inside unsuspecting victims' browsers.

## 5) Types of XSS

### 5.1) Reflected XSS

**Reflected XSS** is a type of Cross site scripting vulnerability in which malicious script is embedded within a request such as **URL parameter**, **search field**, **form field** and is immediately reflected back in the server's response without sanitizing/ validating shown in user browser. As this is not persistent so attacker has to trick the victim by

any mean for attack to succeed common one is **social engineering** . Once the victim's browser executes the script only that specific user's browser is compromised.

#### **Example Payload**

```
https://www.asadweb.com/search?q=<Script>alert('XSS')</Script>
```

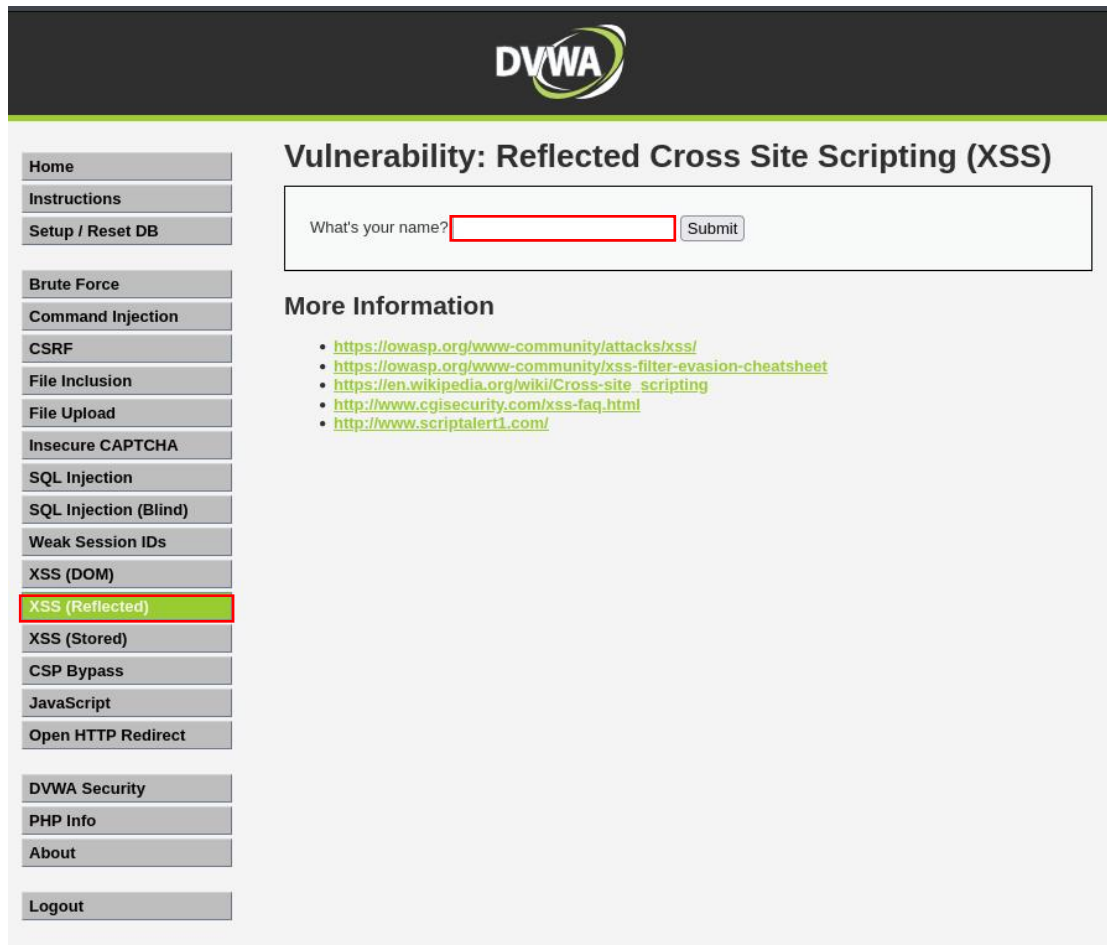
## **5.2) Stored XSS**

**Stored XSS (Persistent XSS)** is the most dangerous form of cross-site scripting. It happens when unvalidated user input is permanently saved in a database (e.g., comments, profiles). When other users visit the page, the stored malicious script executes automatically in their browsers. Unlike Reflected XSS, no phishing link is needed; anyone viewing the affected page is instantly compromised.

#### **Example Payload**

```
<script>alert('Stored XSS - Session Hijacked!');</script>
```

## 6) Reflected XSS Practical



The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. The top header features the DVWA logo. On the left, a sidebar menu lists various security modules, with 'XSS (Reflected)' currently selected and highlighted in green. The main content area is titled 'Vulnerability: Reflected Cross Site Scripting (XSS)'. It contains a form with the label 'What's your name?' followed by a text input field and a 'Submit' button. Below the form, a section titled 'More Information' provides a list of external links related to XSS attacks and defenses.

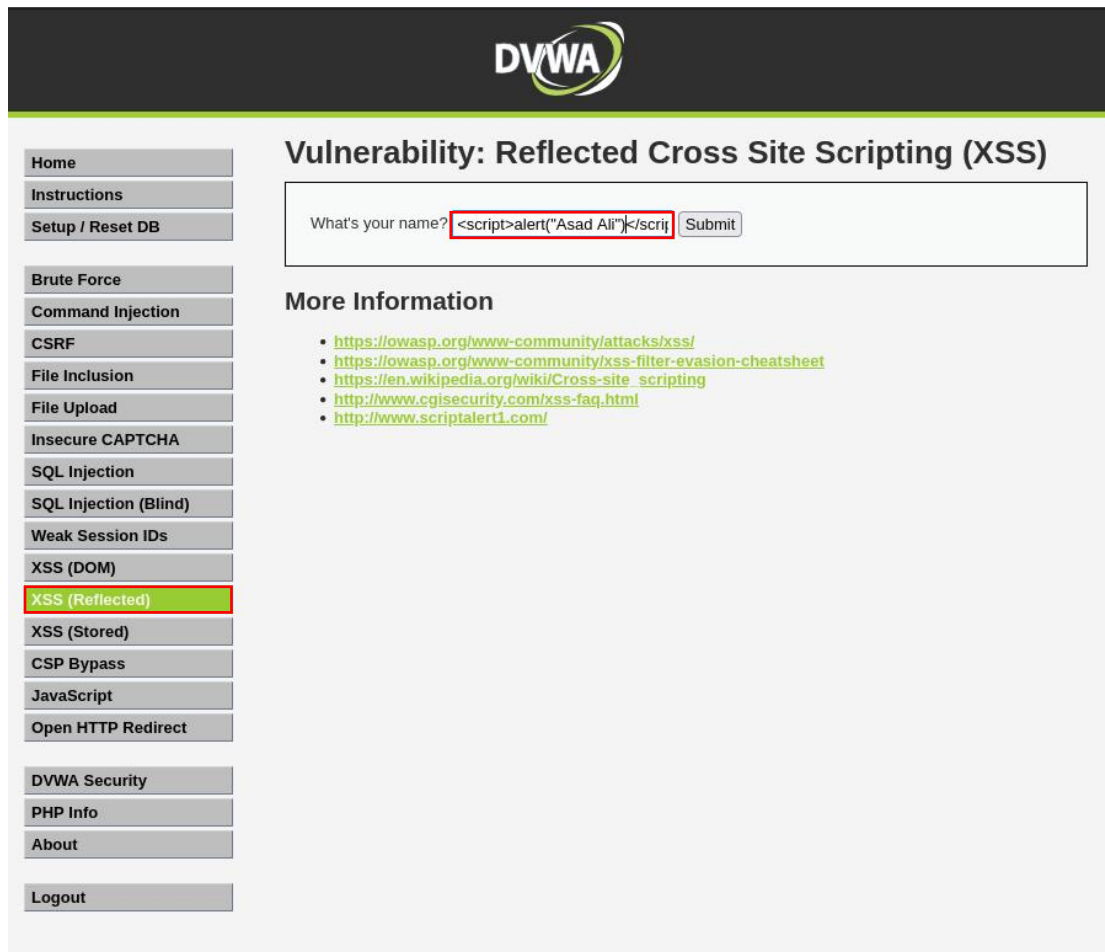
**Vulnerability: Reflected Cross Site Scripting (XSS)**

What's your name?

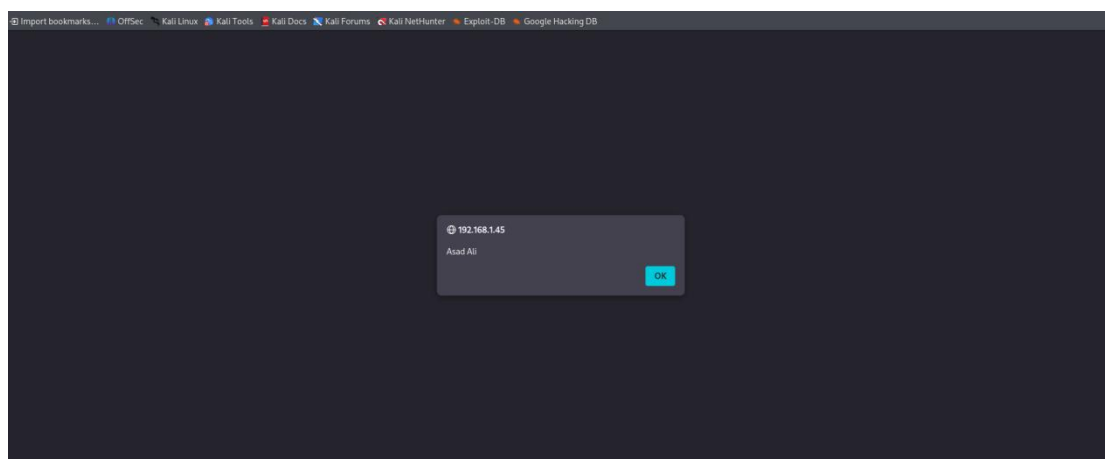
**More Information**

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- [https://en.wikipedia.org/wiki/Cross-site\\_scripting](https://en.wikipedia.org/wiki/Cross-site_scripting)
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

In the above figure it is shown that after login into DVWA application and setting the security level to **Low**. The **Reflected XSS** was selected from the DVWA side menu. As it contains a input field asking what's your name. This XSS module is designed to demonstrate how a web applications can become vulnerable when user input is reflected back to the browser without proper input validation.

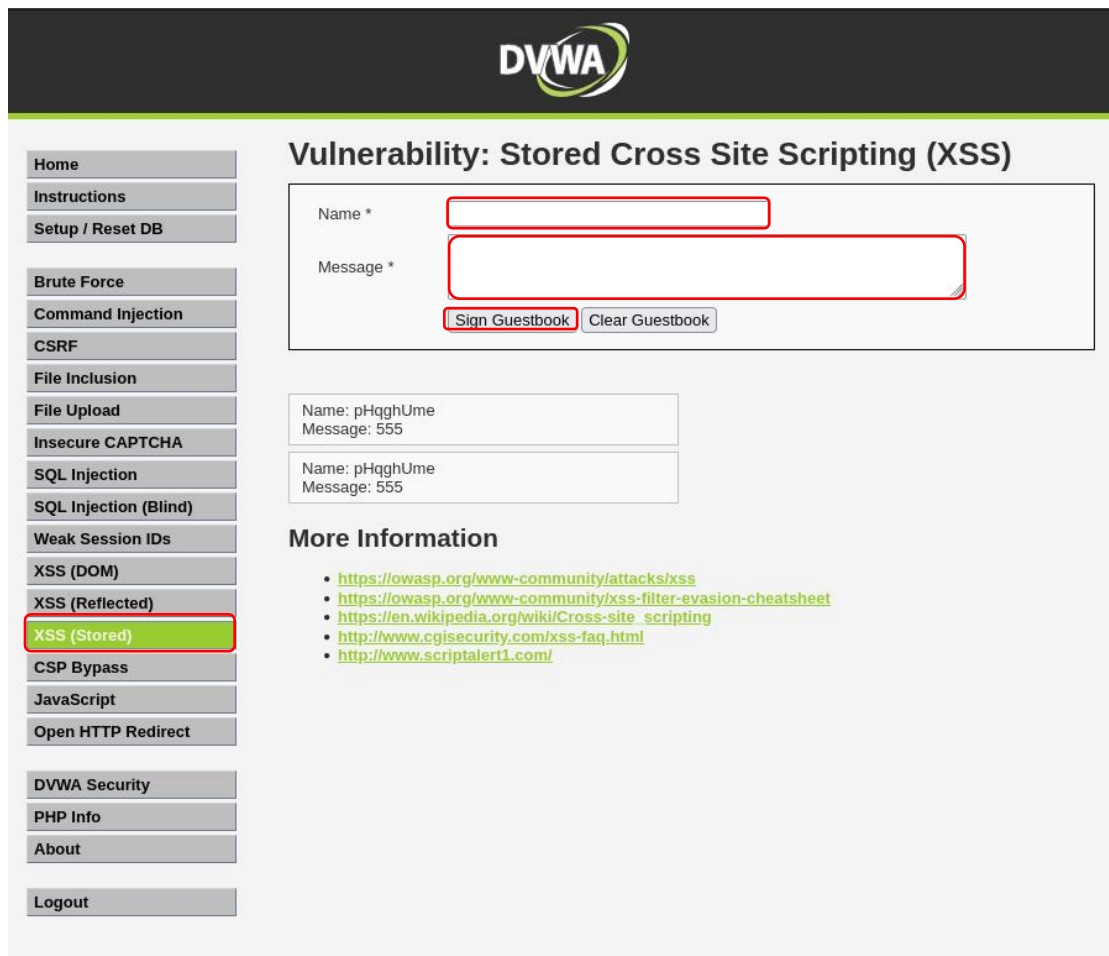


As shown in the **above figure** the following javascript payload: `<Script>alert("Asad Ali")</Script>` is entered into the input field provided to test for Reflected XSS Page.



As Shown in the **above figure** javascript Payload is submitted and malicious script is injected successfully in the browser the response is immediately reflected in the user's browser in the form of javascript alert box displaying the test "Asad Ali". This confirms that the presence of **Reflected XSS**.

## 7) Stored XSS Practical



**DVWA**

**Vulnerability: Stored Cross Site Scripting (XSS)**

Name \*

Message \*

Name: pHqghUme  
Message: 555

Name: pHqghUme  
Message: 555

**More Information**

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- [https://en.wikipedia.org/wiki/Cross-site\\_scripting](https://en.wikipedia.org/wiki/Cross-site_scripting)
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

In the above Picture **Stored XSS** was selected from the left menu of **DVWA** the page appeared which contain two input fields Name and Message. Basically a malicious Javascript code will be executed in the Message input box and then a **sign in guestbook** button will be pressed to store in the Database.



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Open HTTP Redirect

DVWA Security

PHP Info

About

Logout

## Vulnerability: Stored Cross Site Scripting (XSS)

Name \*

Asad Ali

Message \*

<Script>alert("done")</Script>

Sign Guestbook

Clear Guestbook

Name: pHqghUme

Message: 555

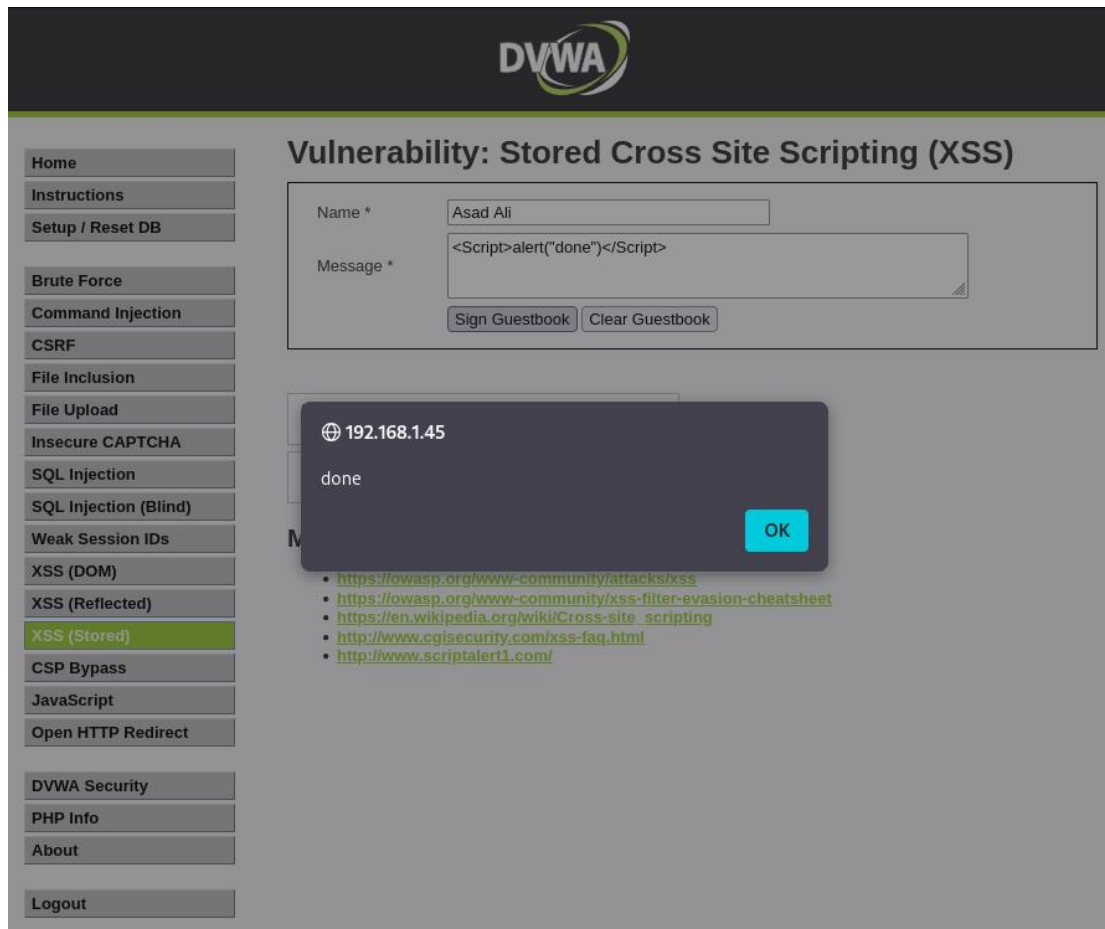
Name: pHqghUme

Message: 555

### More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- [https://en.wikipedia.org/wiki/Cross-site\\_scripting](https://en.wikipedia.org/wiki/Cross-site_scripting)
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

In the above Picture it is shown that following javascript Payload is used :  
<Script>alert("done")</Script> this payload will be stored in the Guestbook database.



In the **above picture** it is clearly shown that the Following javascript Payload: `<Script>alert("done")</Script>` was stored successfully in the guestbook database. Unlike Reflected XSS, the malicious script is saved permanently and executes automatically whenever the vulnerable page is viewed. This makes the Stored XSS more dangerous as compare to **Reflected XSS** because it can affect every user who visits the compromised page.

## 8) Difference between Reflected XSS & Stored XSS

| S.No | Reflected XSS                                                               | Stored XSS                                                                                           |
|------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1    | The malicious script is immediately reflected in the victim's browser       | The malicious script is stored in the applications database then reflect to users.                   |
| 2    | It usually works only when attacker sends malicious link to specific users. | It automatically affects once malicious script stored in DB all users who interacts with application |
| 3    | It is Less dangerous than Stored XSS                                        | It is very dangerous because at once many user get affected                                          |
| 4    | Payload is temporary because it is not saved in a database.                 | Payload is permanent because it is saved in DB.                                                      |
| 5    | Example: Search fields or URL parameters                                    | Example: Comments, guestbooks, forums                                                                |
| 6    | Reflected XSS also known as                                                 | Stored XSS also known as <b>Persistent XSS</b> .                                                     |

|  |                     |  |
|--|---------------------|--|
|  | Non-persistent XSS. |  |
|--|---------------------|--|

## 9) Impact of XSS

Cross site Scripting (XSS) vulnerabilities in the application can have serious consequences if not properly addressed affecting both users and organizations. An attacker can execute malicious scripts in users browser and perform malicious things as per his intention The following points mentioned below highlights the major impacts of XSS attacks

- **Session Hijacking:** Attackers can steal session cookies and impersonate legitimate users.
- **Credentials theft:** Malicious scripts can capture usernames, passwords and other sensitive information.
- **Account Takeover:** Stolen session tokens may allow attackers to gain unauthorized access to user accounts.
- **Malware Distribution:** XSS can be used to redirect users to malicious websites or download malwares.
- **Website Defacement:** Attackers can modify page content and display misleading or malicious information.
- **Phishing and KeyLogging:** Attackers can redirect users to phishing pages to steal sensitive credentials also attacker can plant keylogger to record key stroke of users which include password and card numbers.

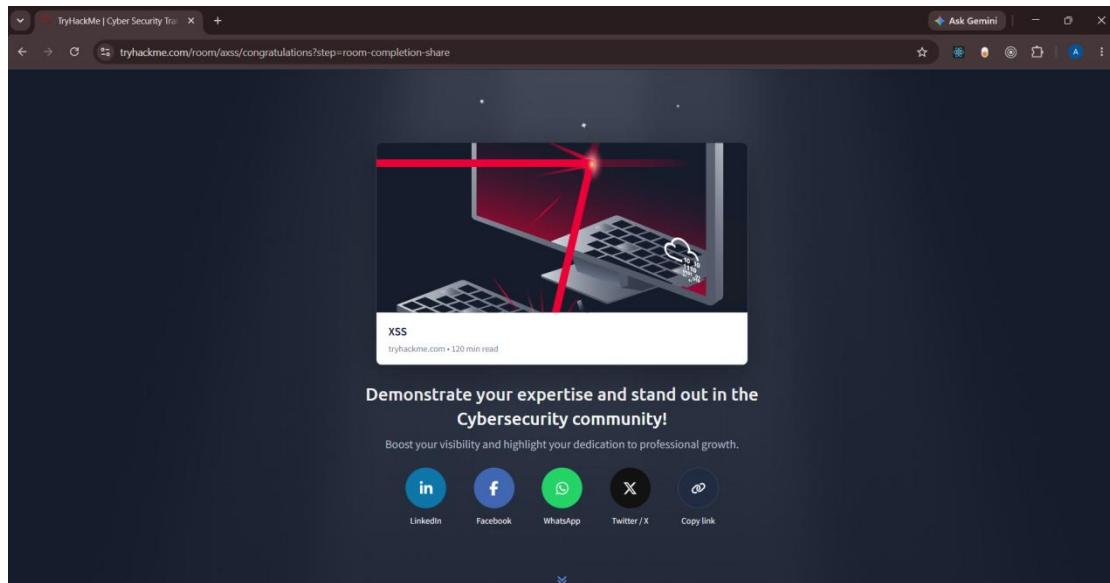
## 10) Remediation of XSS

Cross site Scripting (XSS) vulnerabilities can be prevented by implementing proper security practice during application development. The following measures help to protect applications from XSS attack.

- **Input Validation:** Users input should always be validated before processing. Applications should accept only expected and safe data formats.
- **HTML Sanitization:** Applications should remove or sanitize dangerous HTML tags and JavaScript code from user inputs.
- **Output Encoding:** User generated data should be properly encoded before displaying it on web pages . This prevents browsers from executing injected scripts.
- **Content Security Policy:** Implement CSP headers to restrict the execution of unauthorized scripts.

- **HTTPOnly and Secure Cookies:** Configure cookies with HTTPOnly and secure flags to reduce risk of session theft.
- **Regular Security Testing:** Conduct continuous vulnerability assessment and penteration testing to identify and fix vulnerabilities before attacker exploit them.

## 11) TryHackMe Room XSS Practical



The above figure shows the completion of the **XSS** room on TryHackMe, which helped me improve my understanding of **Cross-Site Scripting (XSS)**, its terminology, types, causes, and potential impact. The room provided practical experience with **Reflected XSS** and **Stored XSS** through vulnerable web applications, while **DOM-based XSS** was covered theoretically. I also learned how XSS payloads interact with different contexts, such as HTML tags and JavaScript, and gained an understanding of basic payload evasion techniques.

## 12) Conclusion

This practical task demonstrated the process of identifying and exploiting Reflected and Stored XSS vulnerabilities in DVWA and TryHackMe within a controlled lab environment. The testing showed how improper input validation and lack of output encoding allow malicious scripts to execute in users' browsers. This exercise improved understanding of XSS risks and secure coding practices. Implementing proper input validation, output encoding, and CSP headers can effectively prevent XSS vulnerabilities in real-world applications